

Lusser’s Law for Agents: The Compounding-Reliability Gap in Long-Horizon Agent Harnesses

Anas Siddiqui

Abstract—Capability metrics for long-horizon AI agents conceal a large gap between the task length at which an agent succeeds half the time and the length at which it succeeds reliably. The 50% time horizon of frontier agents has doubled approximately every seven months, yet the 80% horizon is one quarter to one sixth of it, and identical model weights vary by 9.5–20.8 percentage points in task success across execution harnesses. This paper models a harnessed agent loop as a series reliability system with per-step error, verification sensors characterized by recall r and false-alarm rate f , bounded retries, context degradation, lossy context resets, and verified checkpoints. Seven results are derived: (i) the ratio of trust horizon to capability horizon is invariant to model quality under constant hazard; (ii) Gamma-distributed task heterogeneity reproduces the observed ratios of 1/4 to 1/6 and the declining hazard rates recently reported for agents, without within-task learning; (iii) a sensor multiplies the trust horizon by $G \approx (1 - f)/(1 - r)$; (iv) retries against a stochastic verifier convert detected failures into silent ones (error laundering); (v) the optimal context-reset interval obeys a square-root law analogous to Young’s checkpoint interval; (vi) verified checkpoints reduce expected rerun cost from exponential to linear in task length; and (vii) a closed-form Trust Horizon Equation combines these effects. Monte Carlo simulation of approximately 10^6 synthetic tasks agrees with every closed form to within 7%. The model predicts that once verification is strong, the trust horizon scales only with the inverse square root of the verifier miss rate, and handoff fidelity becomes the binding constraint. Six falsifiable hypotheses and a measurement protocol for real agents are specified.

Index Terms—AI agents, harness engineering, large language models, reliability engineering, survival analysis, software verification, checkpointing.

I. INTRODUCTION

IN reliability engineering, the reliability of a system whose components must all function is the product of the component reliabilities, a relation often attributed to Robert Lusser [1]. Ten components that each function with probability 0.95 yield a system that functions with probability 0.60. This relation motivates redundancy in avionics, retries and idempotency in distributed systems, and the “nines” vocabulary of site reliability engineering [2].

An AI agent executing a long task is a series system. Every step, whether reading a file, invoking a tool, editing code, or selecting the next action, must succeed or be detected and repaired for the task to succeed. An agent with 99% per-step accuracy therefore completes a 100-step task with probability

0.37 and a 500-step task with probability below 0.01. The model is a component; the loop that executes it is the system.

The engineering of that loop is now termed *harness engineering*: the design of the agent loop, context management, tools, memory, feedback and verification, sandboxing, and checkpoints around a model [3], [4]. Evidence that the harness materially affects outcomes is substantial. Among Terminal-Bench 2.0 leaderboard submissions compiled by Guo *et al.*, Claude Opus 4.6 scores 58.0% in one public harness and 76.4% in another [5]. On SWE-bench Pro, Claude Opus 4.5 scores 45.9% under a standardized scaffold and 55.4% under its vendor’s harness, a 9.5-point range that is about twice the 4.9-point range spanned by six leading models under the standardized scaffold [6].

Headline evaluations report *capability*: the probability that an agent completes a task in one attempt, or the task length completed with probability 0.5. Deployment requires *trust*: completion with high probability on every run. The two diverge sharply. In the first release of METR’s time-horizon study, the 80% horizon of the strongest agent measured was about one quarter of its 50% horizon [7]; the published version reports 80% horizons four to six times shorter than 50% horizons [8]. τ -bench reported that GPT-4o’s single-trial success of approximately 61% on retail tasks fell below 25% when all eight repeated trials were required to succeed [9]. Practitioner reports describe the same pattern as silent degradation in long sessions, summaries that omit critical decisions, and loops that consume resources without progress [10]–[12]. This paper refers to the phenomenon as the *run-versus-trust gap* and addresses three research questions:

- RQ1 What functional form does reliability decay take in a harnessed agent loop, and what determines the gap between capability and trust?
- RQ2 Which harness controls (verification, retries, context resets, and checkpoints) extend the trust horizon, by how much, at what cost, and how do they interact?
- RQ3 Which measurements on real agents would confirm or refute the answers?

The contributions are as follows.

- 1) A reliability model of the harnessed agent loop in which each harness component corresponds to one parameter: per-step error ε_0 , sensor recall r and false-alarm rate f , retry budget m , context-degradation rate γ , handoff loss ρ , and reset interval L (Section IV).
- 2) An invariance result for the trust-to-capability ratio, and

A. Siddiqui is with WebridgeAI Research, Calgary, AB, Canada.

Manuscript version 0.1, September 2026. This manuscript is a preprint and has not been peer reviewed.

a frailty account that reproduces the observed ratio and declining population hazards without assuming within-task improvement (Sections IV-B–IV-C).

- 3) Closed forms for the verification gain, the error-laundering effect of retries, the square-root law for context resets, and the cost reduction from checkpoints (Sections IV-D–IV-G).
- 4) The Trust Horizon Equation and a corollary predicting a transition from verification-limited to memory-limited reliability (Section IV-H).
- 5) A Monte Carlo study that validates each closed form and characterizes interactions between controls (Section V).
- 6) Six falsifiable hypotheses and a measurement protocol (Section VII).

Scope: The results are analytical and simulated. No claim is made that any deployed agent has the parameter values used in the simulations. The contribution lies in the functional forms of the predictions (which levers act multiplicatively, which exhibit square-root returns, and which interact), each of which is empirically testable.

II. BACKGROUND AND RELATED WORK

A. Harness Engineering

OpenAI’s report on building an internal beta product of roughly one million lines with no manually written code describes the engineers’ primary job as designing environments, specifying intent, and building feedback loops rather than writing code, and calls this work harness engineering [3]. Böckeler divides a harness into *guides*, feedforward controls such as instructions, architecture maps, and tool schemas that raise the probability of a correct action, and *sensors*, feedback controls such as tests, linters, and review agents that observe outputs and enable self-correction [4]. Böckeler and Ford further argue that computational (deterministic) sensors give more assurance than inferential (model-based) ones wherever a check can be made objective [13]. Anthropic’s engineering guidance addresses composable agent patterns [14], context engineering [15], tool design [16], and a two-stage harness for long-running tasks in which an initializer prepares the environment and a coding agent makes incremental, tested, committed progress recorded in a progress file [17]. Increasing co-training of models with specific harnesses couples the two, so a model may behave differently under an unfamiliar harness [18]. Guo *et al.* [5] survey agent systems through a model–harness lens and decompose the harness into six runtime responsibilities. Zhang *et al.* [6] treat the harness as the controller of a closed-loop system in which the language model acts as a stochastic policy, argue that harness configuration is the binding constraint on performance for frontier-comparable models, and propose disclosure standards. The present work is complementary: it adopts a reliability and survival-analysis formulation that yields closed-form horizons and quantitative predictions for individual harness controls.

B. Measuring Agent Reliability

The $\text{pass}@k$ metric measures the probability that at least one of k attempts succeeds [19]. The pass^k metric introduced

with τ -bench measures the probability that all k attempts succeed [9]. METR’s time horizon is the human-time length of tasks completed with a given probability, estimated with a logistic model in log task length [8]. Ord showed that these data are approximately consistent with a constant hazard per unit of human-equivalent work [20]. Hamilton subsequently fitted Weibull survival models to the same data and found shape parameters $k < 1$ for every agent, between 0.6 and 0.9 for state-of-the-art models and approximately 0.37 for humans, indicating hazard that declines over the course of a task [21]. Ord summarizes the agent values as clustered around 0.6 and notes that Weibull and log-logistic fits differ by a factor of about 20 in their implied 99% horizons [22]. The Holistic Agent Leaderboard reports accuracy–cost trade-offs across models and scaffolds [23].

C. Verification and Self-Correction

Intrinsic self-correction without external feedback often fails to improve, and can degrade, reasoning performance [24]. External verification is more effective: a trained verifier used for trajectory selection improved SWE-bench Verified resolution by 11.4 percentage points in SWE-Gym [25], and weak verifiers are frequently the limiting factor in self-correction [26]. Wei’s verifier’s rule holds that the ease of training AI for a task is proportional to the task’s verifiability [27]; Karpathy makes a related argument about which work is automated first [28].

D. Context Degradation

Model accuracy decreases as input length grows, even for simple retrieval [29], [30]. A study of 18 models found performance becoming increasingly unreliable with input length at fixed task difficulty [30]. Harnesses mitigate this with compaction (summarizing history) and resets (restarting from a structured handoff). One practitioner analysis of long coding sessions found that tool results made up 68% of the context and that compaction discarded them irrecoverably [10]. Anthropic reports that compaction alone is insufficient for multi-session tasks and structures such work as fresh sessions that resume from a progress file and version-control history [17].

E. Reliability Engineering Foundations

Three classical results are used. The reliability of a series system is the product of its component reliabilities [1]. In survival analysis, a population whose members differ in individual hazard (frailty) exhibits declining population hazard even when each individual hazard is constant, because frail members fail first [31]. In high-performance computing, the optimal checkpoint interval balances checkpoint cost against failure rate and follows a square-root law [32], [33].

III. PROBLEM STATEMENT

Agent reliability decays approximately geometrically with task length, so the horizon at which an agent can be trusted lags far behind the horizon at which it is capable. It is not known which harness controls most efficiently reduce this decay, how they interact, or what each is worth per unit cost.

TABLE I
EVIDENCE FOR THE RUN-VERSUS-TRUST GAP. FIGURES AS REPORTED IN THE CITED SOURCES

Phenomenon	Evidence	Source
Harness-induced variance	Identical weights: 58.0% vs. 76.4% (Claude Opus 4.6) and 59.4% vs. 80.2% (Gemini 3.1 Pro) among Terminal-Bench 2.0 leaderboard entries; 45.9% vs. 55.4% (Claude Opus 4.5) on SWE-bench Pro	[5], [6]
Capability vs. trust	Claude 3.7 Sonnet: 59 min at 50% vs. 15 min at 80% reliability (first release); 80% horizons 4–6× shorter than 50% horizons (published version)	[7], [8]
Declining hazard	Weibull shape $k < 1$ for every agent measured (0.6–0.9 for state-of-the-art models); humans ≈ 0.37	[21], [22]
Inconsistency	GPT-4o, retail: pass ¹ 0.604 and pass ⁴ 0.383 (repository leaderboard); pass ⁸ < 0.25 (original paper, separate run)	[9], [34]
Context degradation	Accuracy decreases with input length across 18 models at fixed difficulty	[29], [30]
Lossy compaction	Tool results made up 68% of context in long coding sessions; compaction discarded them irrecoverably	[10]
Stuck loops	Loop detectors terminate legitimate long runs; a malformed tool result can permanently stall a session	[11], [35]
Verifier leverage	External verifier: +11.4 points; intrinsic self-correction can degrade accuracy	[24], [25]

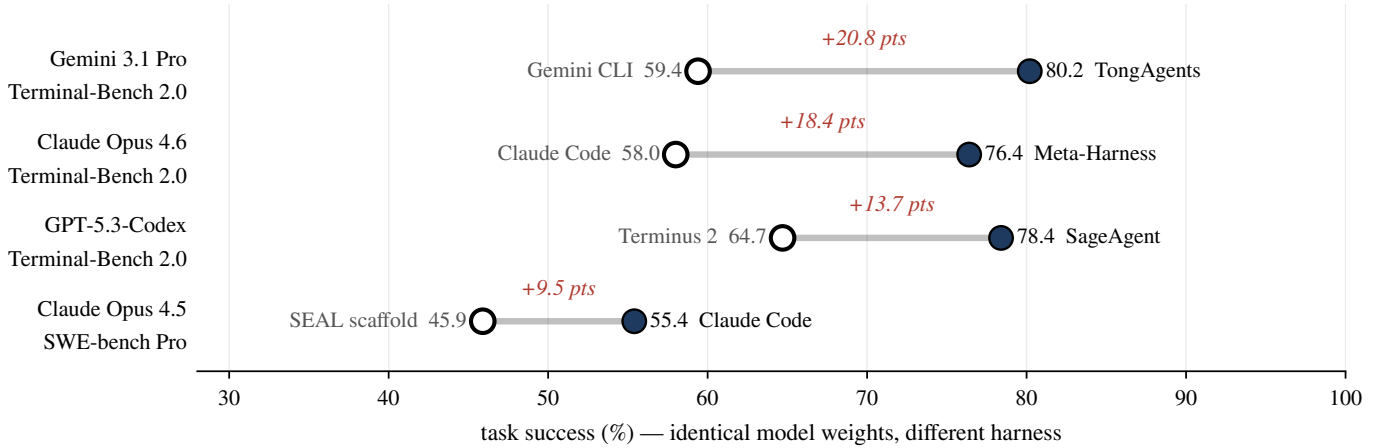


Fig. 1. Task success for identical model weights under different harnesses. Hollow markers: lower-scoring public harness; filled markers: higher-scoring harness. Terminal-Bench 2.0 leaderboard entries as compiled in [5]; SWE-bench Pro data from [6], who draw on a public leaderboard. Differences are observational, not controlled ablations.

Fig. 1 summarizes harness-induced variance: for fixed weights, harness choice changes task success by 9.5 to 20.8 percentage points. These comparisons are observational, since public harnesses differ along many dimensions simultaneously; they establish that the harness matters but not which component is responsible. The capability–trust gap is quantified by METR’s data and appears in repeated-trial form in τ -bench (Section IV-C). The failure modes most frequently reported by practitioners, namely context loss, silent drift, stuck loops, and brittle recovery, are failures of the loop rather than of any single model invocation (Table I).

Prior work measures the decay curve [8], measures inconsistency [9], or documents harness variance [6], [23]. No existing work decomposes the decay into harness-controllable factors or predicts the effect of a specific harness modification on the trust horizon. Practitioner guidance to add sensors, compact less, or commit progress remains qualitative.

IV. RELIABILITY MODEL OF THE HARNESSSED AGENT LOOP

A. Model Definition

A task comprises n steps, where a step is a unit of progress on which the remainder of the task depends. In the protocol of Section VII, steps are operationalized as verifiable intermediate states. Fig. 2 maps each harness component to its model parameter.

Definition 1 (Harnesssed loop): At each step the agent makes up to m attempts. An attempt made s steps after the most recent context reset is incorrect with probability $\varepsilon(s) = \varepsilon_0(1 + \gamma s)$. A sensor flags an incorrect attempt with probability r (recall) and a correct attempt with probability f (false-alarm rate). An unflagged attempt is accepted. A flagged attempt is retried if attempts remain and is otherwise escalated, which constitutes a detected failure. An accepted incorrect attempt is a silent error. The context may be reset every L steps; each reset loses task-critical state with probability ρ (handoff loss), which also constitutes a silent error. A task succeeds if all n steps are accepted correctly.

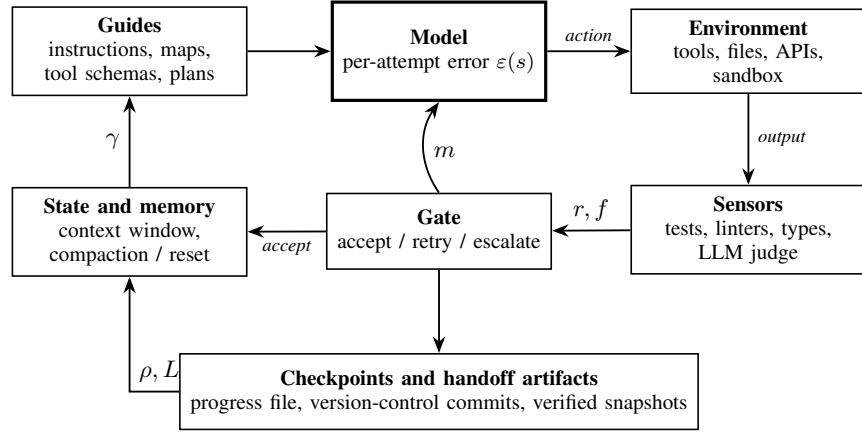


Fig. 2. Harnessed agent loop and the model parameter associated with each component. Guides and context determine the per-attempt error ε ; sensors have recall r and false-alarm rate f ; the gate permits up to m attempts per step; context degrades at rate γ ; resets every L steps lose task-critical state with probability ρ .

TABLE II
PER-STEP RELIABILITY $p = \alpha^{1/n}$ REQUIRED FOR END-TO-END
RELIABILITY α OVER n STEPS

	$n = 10$	$n = 100$	$n = 1000$
$\alpha = 0.50$	93.30%	99.31%	99.931%
$\alpha = 0.80$	97.79%	99.78%	99.978%
$\alpha = 0.99$	99.90%	99.990%	99.9990%

With one attempt, no sensor, and no degradation, the model reduces to n Bernoulli trials with success probability $p = 1 - \varepsilon_0$.

Definition 2 (Trust horizon): For a reliability target $\alpha \in (0, 1)$, the trust horizon n_α is the largest task length for which the success probability is at least α . The capability horizon is $n_{0.5}$; unless otherwise specified, the trust horizon refers to $n_{0.8}$. The hazard is the per-step failure rate $h = -\frac{d}{dn} \ln R(n)$, where $R(n)$ is the success probability for an n -step task.

The independence assumptions in Definition 1 are relaxed across tasks in Section IV-C and across retries in Section IV-E.

B. Series Reliability and Invariance of the Gap

Proposition 1 (Invariance): Under constant per-step error ε , $R(n) = (1 - \varepsilon)^n$, and

$$n_\alpha = \frac{\ln(1/\alpha)}{-\ln(1 - \varepsilon)} \approx \frac{\ln(1/\alpha)}{\varepsilon}, \quad \frac{n_\alpha}{n_\beta} = \frac{\ln \alpha}{\ln \beta}. \quad (1)$$

In particular, $n_{0.8}/n_{0.5} = \ln 0.8 / \ln 0.5 \approx 0.322$ and $n_{0.99}/n_{0.5} \approx 0.0145$, independent of ε .

Horizons therefore scale as $1/\varepsilon$: halving per-step error doubles every horizon. However, under memoryless failure no improvement in the model alters the ratio of trust horizon to capability horizon, as Ord observed for METR's data [20]. The ratio changes only if the loop ceases to be memoryless, which is the function of harness controls. Fig. 3 and Table II quantify the per-step reliability required.

C. Task Heterogeneity and Declining Hazard

METR's first release gives a ratio of 0.25 for Claude 3.7 Sonnet (15 versus 59 min) rather than 0.32 [7], the published

version reports ratios of about 1/4 to 1/6 [8], and Weibull fits indicate hazard that declines over a task [21], [22]. One interpretation is that agents improve during a task. Proposition 2 provides an alternative based on heterogeneity across tasks.

Proposition 2 (Frailty): Suppose each task has a constant hazard λ per unit length and $\lambda \sim \text{Gamma}(k, \beta)$ across tasks, with shape k and rate β . Then the population survival function and hazard are

$$S(t) = \left(1 + \frac{t}{\beta}\right)^{-k}, \quad h(t) = \frac{k}{\beta + t}, \quad (2)$$

so the population hazard declines although no individual hazard does. Horizons satisfy $T_\alpha = \beta(\alpha^{-1/k} - 1)$ and

$$\frac{T_{0.8}}{T_{0.5}} = \frac{1.25^{1/k} - 1}{2^{1/k} - 1}, \quad (3)$$

which equals 0.25 exactly at $k = 1$, where S is log-logistic with unit shape, equals 1/5 and 1/6 at $k \approx 0.56$ and 0.42, and tends to 0.322 as $k \rightarrow \infty$.

This effect is well established in demography [31]; to the author's knowledge it has not previously been applied to agent evaluation. The distinction matters because the two interpretations imply different harness designs. If agents improve within a task, long tasks are less hazardous than population curves suggest and persistence is rewarded. If the decline reflects selection, whereby difficult tasks fail early, each difficult task retains its full hazard and retries are less effective than they appear. Population survival curves cannot distinguish the two; repeated trials of the same task can.

Proposition 3 (pass^k under heterogeneity): Let the per-trial success probability of a task be $\theta \sim \text{Beta}(a, b)$ across tasks, with trials conditionally independent given θ . Then

$$\text{pass}^k = \mathbb{E}[\theta^k] = \prod_{j=0}^{k-1} \frac{a + j}{a + b + j}, \quad (4)$$

$$\text{pass}@k = 1 - \prod_{j=0}^{k-1} \frac{b + j}{a + b + j}, \quad (5)$$

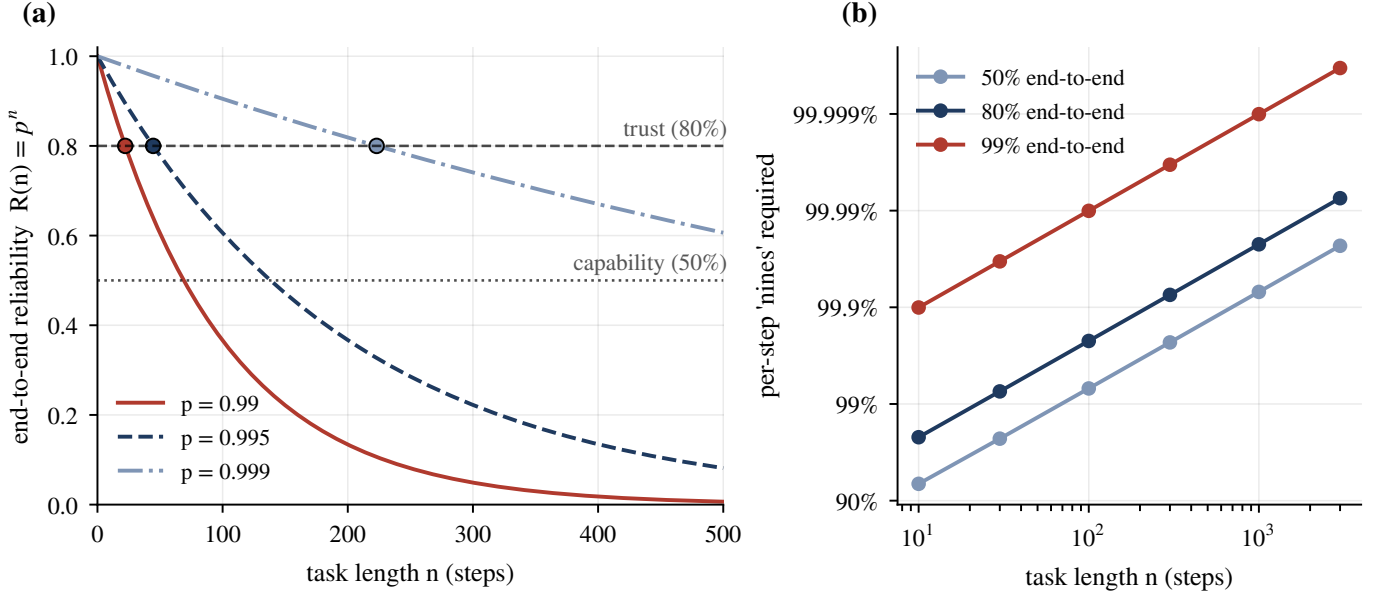


Fig. 3. Series reliability of agent loops. (a) End-to-end reliability $R(n) = p^n$ for three per-step reliabilities; circles mark the 80% trust horizon ($n_{0.8} = 22, 44, \text{ and } 223$). (b) Per-step reliability required to reach a given end-to-end reliability; a 1000-step task at 99% end-to-end reliability requires 99.999% per step.

TABLE III
BETA-HETEROGENEITY FITS TO τ -BENCH LEADERBOARD VALUES [34]

Agent, domain	p^1	p^4	i.i.d. p^4	κ	p^8	$p@8$
GPT-4o, retail	0.604	0.383	0.133	0.51	0.30	0.87
Claude 3.5 S., retail	0.692	0.462	0.229	0.45	0.36	0.94
Claude 3.5 S., airline	0.460	0.225	0.045	0.47	0.15	0.79

p^k : pass^k ; $p@8$: $\text{pass}@8$. p^1 and p^4 are leaderboard values for tool-calling agents (Claude 3.5 Sonnet: 2024-10-22 release) on the original task set, which was subsequently revised; p^8 and $p@8$ are model predictions.

and the intra-task correlation $\kappa = 1/(a + b + 1)$ is the fraction of outcome variance attributable to task identity. Homogeneous tasks correspond to $\kappa \rightarrow 0$, for which $\text{pass}^k \rightarrow p^k$.

Fitting (4) and (5) to the ($\text{pass}^1, \text{pass}^4$) pairs on the τ -bench repository leaderboard [34] yields $\kappa \approx 0.45\text{--}0.51$ (Table III): approximately half of outcome variance is explained by task identity rather than run-to-run variation. The fit predicts $\text{pass}^8 = 0.30$ for GPT-4o on retail tasks, compared with 0.018 under independence. The original paper reports pass^8 below 0.25 [9], an order of magnitude closer to the heterogeneous prediction but somewhat lower; because that value comes from a separate run ($\text{pass}^1 = 0.612$), the comparison is indicative only. Fig. 5(b) illustrates the consequence. With a perfect selector and independent failures, eight attempts would raise GPT-4o’s retail success to 99.94%; under the fitted heterogeneity they reach 87%. This difference is termed the *retry gap*. The heterogeneity that produces apparently declining hazard also makes retries less effective than independent-trial calculations indicate.

D. Verification With Bounded Retries

Fig. 6 represents one step as a Markov chain. Each attempt terminates in one of three states with probabilities

$$\begin{aligned} a &= (1 - \varepsilon)(1 - f) \quad (\text{accept, correct}), \\ b &= \varepsilon(1 - r) \quad (\text{accept, incorrect}), \\ c &= (1 - \varepsilon)f + \varepsilon r \quad (\text{reject}). \end{aligned} \quad (6)$$

Proposition 4 (Verified step): With up to m independent attempts, a step terminates correct, silently incorrect, or escalated with probabilities

$$P_{\text{ok}} = a \frac{1 - c^m}{1 - c}, \quad P_{\text{sil}} = b \frac{1 - c^m}{1 - c}, \quad P_{\text{esc}} = c^m, \quad (7)$$

with $(1 - c^m)/(1 - c)$ attempts in expectation. As $m \rightarrow \infty$ the residual silent-error rate per step is $\tilde{\varepsilon} = b/(a + b)$; for small ε ,

$$\tilde{\varepsilon} \approx \varepsilon \frac{1 - r}{1 - f}, \quad G \equiv \frac{\varepsilon}{\tilde{\varepsilon}} \approx \frac{1 - f}{1 - r}. \quad (8)$$

By (1), every trust horizon is multiplied by G .

The verification gain G is determined primarily by recall. A sensor with $r = 0.9$ and $f = 0.05$ multiplies the horizon by 9.5; one with $r = 0.5$ by approximately 2 (Fig. 7). The false-alarm rate enters the gain only through the factor $1 - f$, but has a larger effect through the escalation probability $c^m \approx f^m$: with $m = 1$, every false alarm terminates the task. Section V-D shows that this suffices to make a strong verifier without retries worse than no verifier.

E. Error Laundering Under Retries

Proposition 4 assumes independent retries. In practice, an agent that misinterprets a requirement often does so identically on every attempt. A step is termed *stuck* if every attempt produces the same incorrect output.

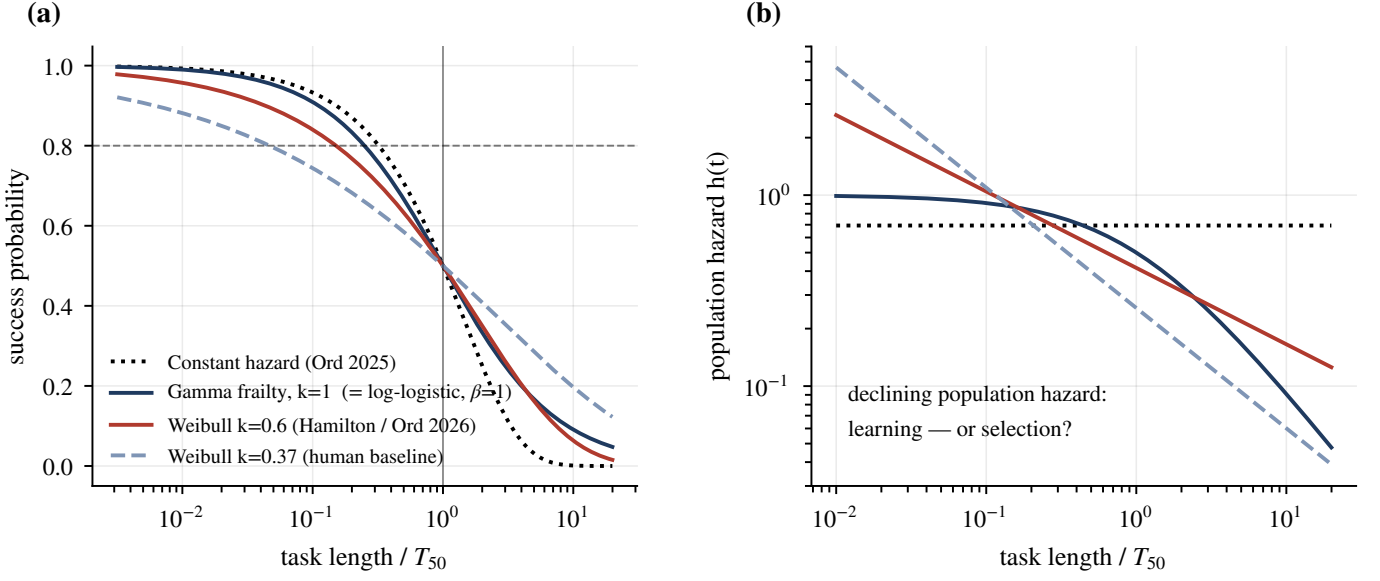


Fig. 4. Reliability-decay shapes normalized to a common 50% horizon. (a) Survival functions; the dashed line marks 80%. (b) Population hazard. Constant hazard is flat; Gamma frailty and Weibull $k < 1$ both decline. The curves agree near T_{50} and differ by more than an order of magnitude in their implied 99% horizons.

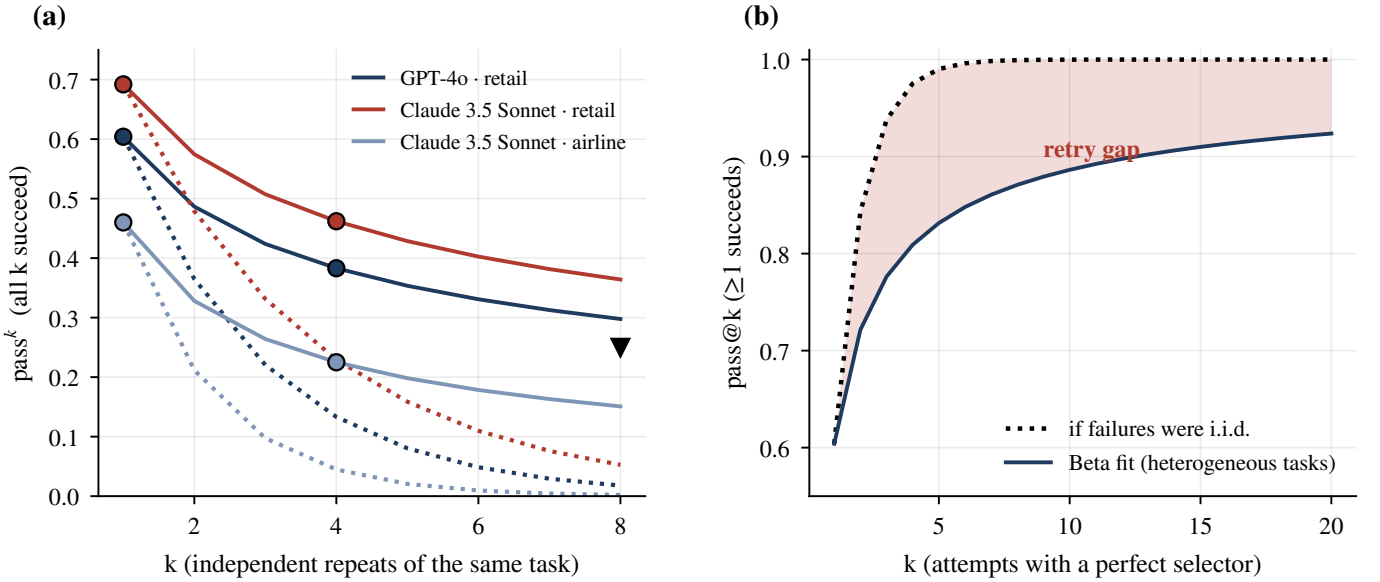


Fig. 5. Consistency under task heterogeneity. (a) pass^k : reported values (circles), Beta fits (solid), and independent-trial prediction p^k (dotted); the triangle marks the reported bound $\text{pass}^8 < 0.25$ for GPT-4o on retail tasks. (b) $\text{pass}@k$ for GPT-4o on retail tasks; the shaded region is the retry gap.

Proposition 5 (Error laundering): Let a step be stuck with probability φ , and let sensor verdicts be independent across attempts, as for a sampled language-model judge or a nondeterministic test. Then

$$P(\text{silent} \mid \text{stuck}) = 1 - r^m \xrightarrow{m \rightarrow \infty} 1. \quad (9)$$

If a silent error costs λ times the value of a success, the marginal value of the j th attempt is proportional to $(1 - \varphi) a c^{j-1} - \lambda \varphi (1 - r) r^{j-1}$. Because $c < r$ in any useful regime, this quantity is negative for all $j > m^*$, where

$$m^* = 1 + \frac{\ln[(1 - \varphi) a / (\lambda \varphi (1 - r))]}{\ln(r/c)}. \quad (10)$$

For a moderately difficult step ($\varepsilon = 0.05$, $r = 0.9$, $f = 0.05$), a 2% probability of being stuck, and silent errors ten times as costly as a success, $m^* \approx 2.7$. The optimal retry budget therefore grows only logarithmically with the stakes. Fig. 8 shows that on ordinary steps retries saturate after two or three attempts, whereas on stuck steps the probability of accepting an incorrect output increases steadily with m .

Remark 1: Equation (9) requires independent verdicts. A deterministic check, such as a type checker or a fixed unit test, either detects a stuck error on every attempt or on none, so retries cannot launder errors past it. This gives a quantitative reason to prefer computational sensors wherever a deterministic

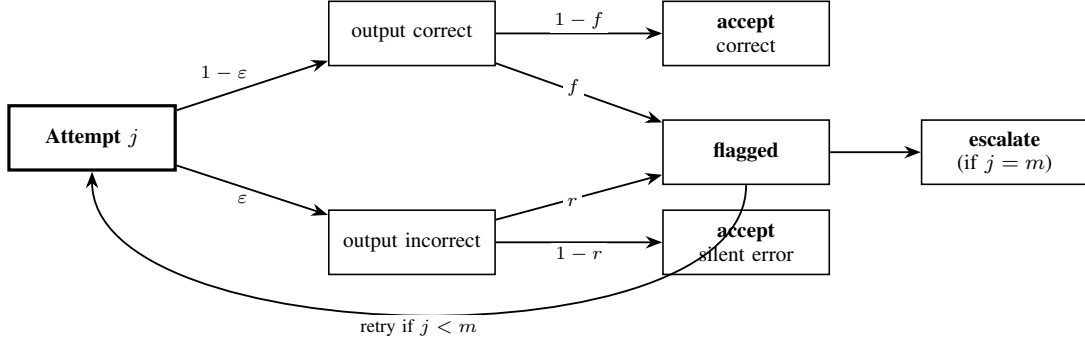


Fig. 6. One verified step as a Markov chain. The step terminates as a correct acceptance, as a silent error, or as an escalation after m flagged attempts.

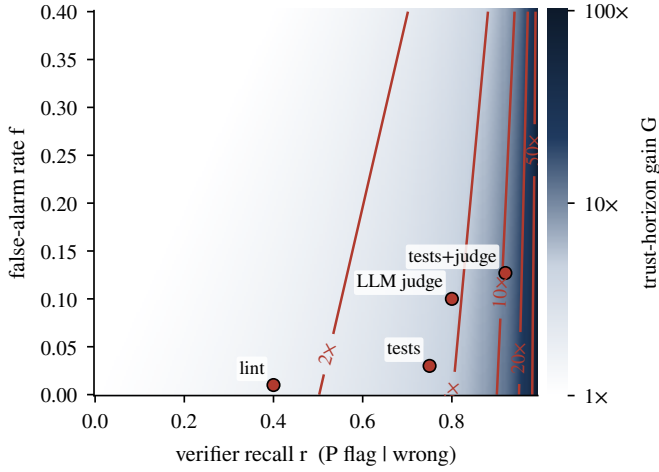


Fig. 7. Trust-horizon gain G for a sensor with recall r and false-alarm rate f (unlimited retries, $\varepsilon = 0.01$). Markers denote the illustrative sensor configurations of Section V-D; their parameters are assumptions.

check exists, consistent with practitioner guidance [13].

F. Context Degradation and a Square-Root Law for Resets

Context degradation is modeled as a linear increase in per-attempt error with steps since the most recent reset, $\varepsilon(s) = \varepsilon_0(1 + \gamma s)$, so the error doubles after $1/\gamma$ steps. Without resets, $-\ln R(n) \approx \varepsilon_0 n + \frac{1}{2}\varepsilon_0\gamma n^2$, and reliability decays as a Gaussian rather than an exponential function of n . Resets bound this growth but risk state loss.

Proposition 6 (Square-root law): With resets every L steps and handoff loss ρ per reset, the per-step hazard is, to first order,

$$h(L) = \varepsilon_0 + \frac{1}{2}\varepsilon_0\gamma(L-1) + \frac{\rho}{L}, \quad (11)$$

which is minimized at

$$L^* = \sqrt{\frac{2\rho}{\varepsilon_0\gamma}}, \quad h(L^*) \approx \varepsilon_0 + \sqrt{2\varepsilon_0\gamma\rho}. \quad (12)$$

Equation (12) has the form of Young's optimal checkpoint interval, $\tau^* = \sqrt{2\delta M}$, for checkpoint cost δ and mean time between failures M [32], [33]. Here the cost of a reset is the probability of losing state in the handoff, and the failure rate is the rate at which degradation accumulates. Fig. 9 confirms the

law in simulation. Two practical consequences follow. First, the hazard is U-shaped in L , so excessively frequent resets are as harmful as infrequent ones. Second, because a verifier rescales ε_0 to $\tilde{\varepsilon}$, adding a sensor shifts the optimum: $L_{\text{verified}}^*/L_{\text{bare}}^* = \sqrt{G}$. With $G = 9.5$ the optimal interval increases from 20 to 62 steps in the reference setting, so a compaction policy tuned before a verifier is introduced is miscalibrated afterwards.

G. Checkpoints and Rerun Cost

Agent errors typically remain silent until checked. If a task is checked only at completion, at cost v , and rerun on failure, the expected work is $(n+v)/p^n$, which is exponential in n .

Proposition 7 (Checkpointing): With per-step success probability $p = e^{-h}$, a perfect segment check of cost v every L steps, and rollback on failure, the expected work is

$$W(n) = \frac{n}{L} \cdot \frac{L+v}{p^L}, \quad L^* = \frac{-v + \sqrt{v^2 + 4v/h}}{2} \approx \sqrt{v/h}, \quad (13)$$

which is linear in n and again exhibits a square-root optimum.

For $p = 0.99$ and $v = 5$, $L^* \approx 20$, and a 1000-step task requires approximately 1530 step-equivalents rather than 2.3×10^7 (Fig. 10). Checkpoints alone do not, however, confer trust. If the segment check has recall $r_s < 1$, silent errors pass it with probability $s = (1-p^L)(1-r_s)/[p^L + (1-p^L)(1-r_s)]$ per accepted segment, and end-to-end reliability remains $(1-s)^{n/L}$, geometric in n . Checkpoints reduce cost; sensors increase trust. The long-running harness described in [17], which commits tested progress to version control and records it in a progress file for subsequent sessions, combines both.

H. The Trust Horizon Equation

Combining Propositions 4 and 6, with ε_0 replaced by the silent-error rate $\tilde{\varepsilon}$ and the reset interval re-optimized, and adding the detected-failure floor due to escalation, yields the central result:

$$n_\alpha^* \approx \frac{\ln(1/\alpha)}{\tilde{\varepsilon} + \sqrt{2\tilde{\varepsilon}\gamma\rho + f^m}}, \quad \tilde{\varepsilon} = \varepsilon_0 \frac{1-r}{1-f}, \quad (14)$$

with optimal reset interval $L^* = \sqrt{2\rho/(\tilde{\varepsilon}\gamma)}$. The three denominator terms represent errors that pass the sensor, the combined cost of context degradation and lossy handoffs, and the false-alarm floor, respectively. The expression is valid in

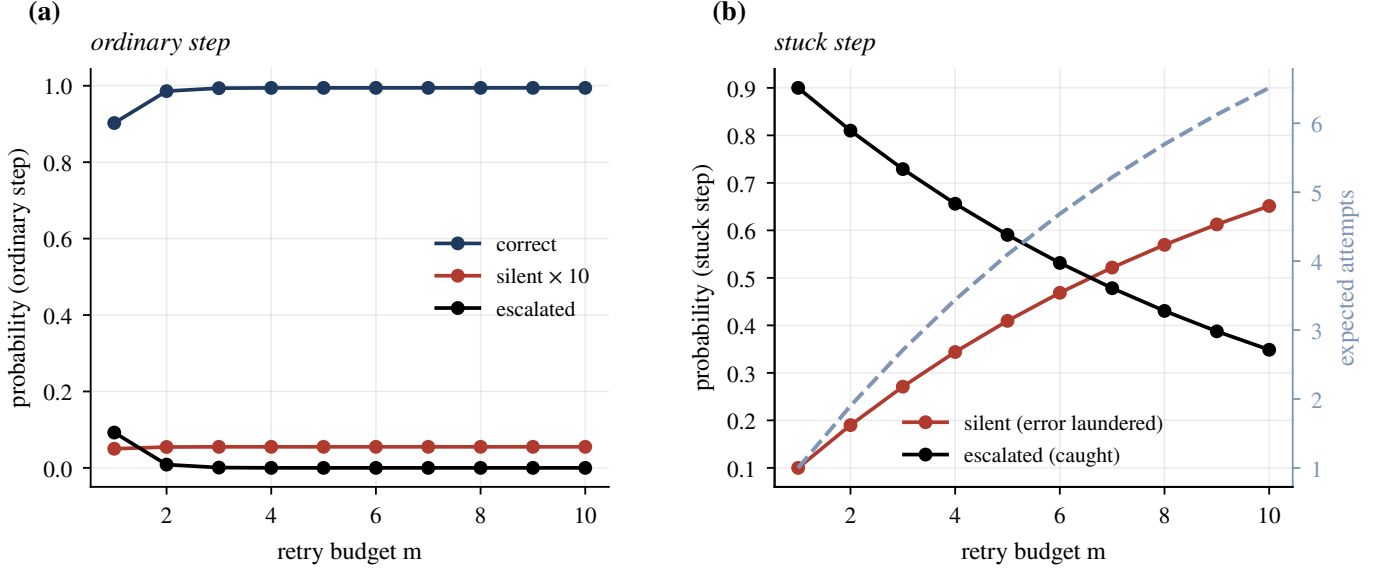


Fig. 8. Retry budgets. (a) Ordinary step ($\varepsilon = 0.05$, $r = 0.9$, $f = 0.05$): success saturates by $m = 3$. (b) Stuck step with a stochastic sensor: each retry increases the probability $1 - r^m$ that an incorrect output is accepted, and the expected number of attempts (dashed, right axis).

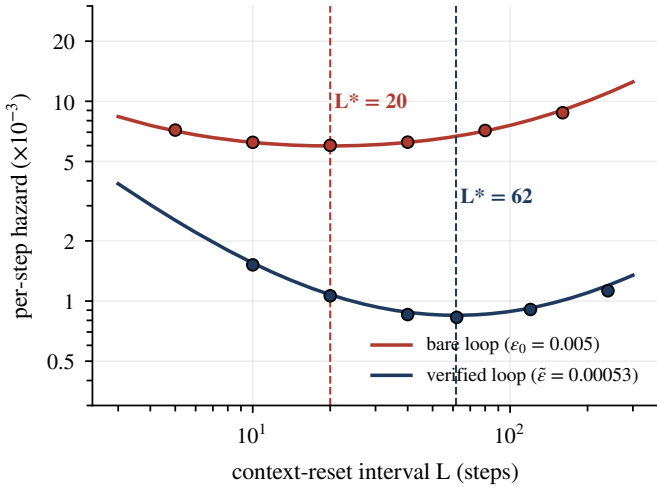


Fig. 9. Per-step hazard versus reset interval for a bare loop ($\varepsilon_0 = 0.005$) and a verified loop ($r = 0.9$, $f = 0.05$), with $\gamma = 0.01$ and $\rho = 0.01$. Lines: (11); markers: simulation (20 000 tasks per point); dashed lines: L^* from (12).

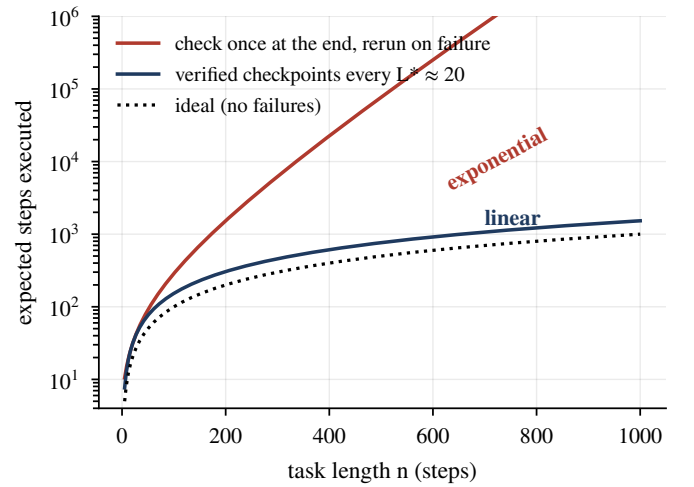


Fig. 10. Expected work to complete a task ($p = 0.99$, check cost $v = 5$). A single terminal check with full reruns grows exponentially; verified checkpoints at $L^* \approx 20$ grow linearly, at approximately $1.5\times$ the ideal.

the small-error regime and uses $c^m \approx f^m$ for $\varepsilon \ll f$. Handoff loss ρ is not discounted by recall, because a step-level sensor evaluates whether an action was correct given its context, not whether the context retains the information required. A preliminary version of the model that discounted ρ by $(1 - r)$ disagreed with simulation by a factor of two, which identified this asymmetry.

Corollary 1 (Two regimes): When $\tilde{\varepsilon} \gg 2\gamma\rho$ the loop is verification-limited: $n_\alpha^* \propto 1/\tilde{\varepsilon} \propto (1 - r)^{-1}$, and doubling sensor quality doubles the horizon. When $\tilde{\varepsilon} \ll 2\gamma\rho$ it is memory-limited: $n_\alpha^* \approx \ln(1/\alpha)/\sqrt{2\tilde{\varepsilon}\gamma\rho} \propto (1 - r)^{-1/2}$, and a tenfold improvement in sensor quality yields approximately a $3.2\times$ improvement in horizon. The regimes meet at $\tilde{\varepsilon} = 2\gamma\rho$.

In the reference setting ($\varepsilon_0 = 0.005$, $\gamma = \rho = 0.01$), the

transition occurs at $r \approx 0.96$ (Fig. 11). Beyond it, the more effective lever is ρ , the fidelity of the artifacts that carry state across a reset. This formalizes current practice in long-running harnesses, which rely on structured progress files, commits after each verified step, and initialization into a known environment state [3], [17].

V. SIMULATION STUDY

A. Simulator

Definition 1 was implemented as a vectorized Monte Carlo simulator in Python and NumPy (Appendix B). Each run simulates between 8000 and 50 000 independent tasks step by step and records the step and type (silent or escalated) of the first failure, so a single run yields the full survival

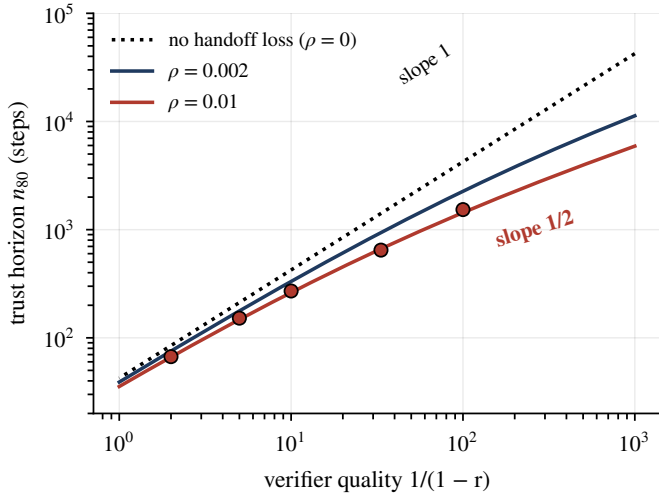


Fig. 11. Trust horizon versus verifier quality $1/(1-r)$ at the re-optimized reset interval. Without handoff loss (dotted) the slope is 1; with $\rho > 0$ it approaches $1/2$. Markers: simulation at $\rho = 0.01$.

TABLE IV
REFERENCE SIMULATION PARAMETERS

Symbol	Value	Meaning
ε_0	0.005	per-attempt error immediately after a reset
γ	0.01	error doubles after 100 steps without reset
ρ	0.01	handoff loss per reset (0.002 in the improved-handoff condition)
r, f	0.9, 0.05	reference sensor recall and false-alarm rate
m	3	retry budget per step
v	0.3	sensor cost per attempt
Tasks	8000–50 000	per condition; 95% bootstrap intervals on $n_{0.8}$ within ± 2 steps for $n_{0.8} < 50$ and $\pm 5\%$ otherwise

curve $R(n)$. Cost is measured in generation-attempt units: one per attempt, plus v per sensor invocation and a fixed charge per reset. Table IV lists the reference parameters, which were selected to be plausible and to expose each effect; they were not estimated from any deployed agent. Approximately 10^6 synthetic tasks were simulated in total. The code is available at <https://github.com/big-oh/trust-horizon>.

B. Validation of Closed Forms

Table V compares each closed-form prediction with simulation. The largest discrepancy, 7%, occurs at the highest recall, where the first-order expansion in Proposition 6 is least accurate.

C. Ablation of Harness Controls

Controls were added individually to a bare loop (Fig. 12). Four findings emerge.

Resets alone do not improve trust. They extend the 50% horizon from 95 to 118 steps but leave the 80% horizon essentially unchanged (38 to 39), because degradation becomes significant only after approximately $1/\gamma = 100$ steps, by which point a bare loop has usually failed. Context management is beneficial only once other controls make long runs survivable.

The sensor is the dominant lever. The reference verifier increases the trust horizon by a factor of 4.8 (38 to 184). This is below the unconstrained gain of 9.5 because degradation is active over the longer horizon and escalations add a failure floor.

Controls must be tuned jointly. Adding resets at the interval optimal for the bare loop ($L = 20$) slightly reduces the trust horizon (184 to 180). Re-tuning to $L^* = 62$, as Proposition 6 predicts, increases it to 224.

Handoff fidelity then becomes binding. A fivefold reduction in handoff loss increases the horizon to 267, more than any further change in reset interval. The complete harness extends the trust horizon by a factor of 7.0.

D. Cost–Reliability Frontier

A 150-step task was simulated under five illustrative sensor configurations (none, linter, unit tests, language-model judge, and tests combined with judge; parameters in Fig. 7), four retry budgets, and resets enabled or disabled, for 34 configurations in total (Fig. 13).

With $m = 1$, success falls to 0–10% for every sensor, because each false alarm over 150 steps terminates the task; a strong judge with $m = 1$ achieves 0%, compared with 26.5% for no sensor. Verification reduces cost per successful task: unit tests with $m = 3$ raise per-step cost by 30% but reduce cost per success by approximately 30%, because fewer runs are wasted. Inexpensive deterministic sensors dominate the lower portion of the frontier: tests achieve 71–75% success at $1.6\times$ the ideal cost per success, whereas the judge alone costs more for comparable success. Combining tests and judge yields the highest success (88.5%) and reduces the share of tasks ending in silent failure from 73.5% to 10.8%, at approximately $2.3\times$ ideal cost.

The numerical values depend on the assumed sensor parameters, but the qualitative structure does not: false alarms require a retry budget, recall determines trust, and sensor cost must be assessed per successful task rather than per step.

VI. IMPLICATIONS FOR HARNESS DESIGN

The model yields seven design principles, each testable under the protocol of Section VII.

- 1) Measure reliability per step as well as per task, and classify each failure as silent or detected; trust is determined by the silent fraction.
- 2) Report $n_{0.8}$ and pass^k alongside $\text{pass}@1$ and $n_{0.5}$; capability metrics cannot reveal the run-versus-trust gap, and pass^k is the only routine metric sensitive to task heterogeneity.
- 3) Prioritize sensor recall, and prefer computational sensors wherever a deterministic check exists; the gain is approximately $(1-f)/(1-r)$, and deterministic checks do not launder errors.
- 4) Pair every sensor with a bounded retry budget of approximately two to four attempts followed by escalation; the optimal budget grows only logarithmically with the stakes.

TABLE V
CLOSED-FORM PREDICTIONS VERSUS MONTE CARLO SIMULATION

Result	Quantity	Setting	Predicted	Simulated
Prop. 1	$R(100)$	$\varepsilon = 0.01$	0.366	0.363
Prop. 2	$T_{0.8}/T_{0.5}$	Gamma frailty, $k = 1$	0.250	0.240
Prop. 4	$n_{0.8}$	$(r, f) = (0.5, 0.02)$	43.4	43
		$(r, f) = (0.8, 0.05)$	105.0	107
		$(r, f) = (0.9, 0.05)$	210.0	209
		$(r, f) = (0.97, 0.10)$	662.8	653
Prop. 5	$P(\text{silent} \mid \text{stuck})$	$r = 0.9, m = 6$	0.469	0.471
Prop. 6	$h(L) \times 10^3$	$L = 5, 10, 20, 40, 80$	7.10, 6.23, 5.97, 6.23, 7.10	7.10, 6.24, 6.01, 6.22, 7.11
Eq. (14)	$n_{0.8}$	$r = 0.5, 0.9, 0.99; \rho = 0.01$	66, 262, 1438	67, 271, 1533

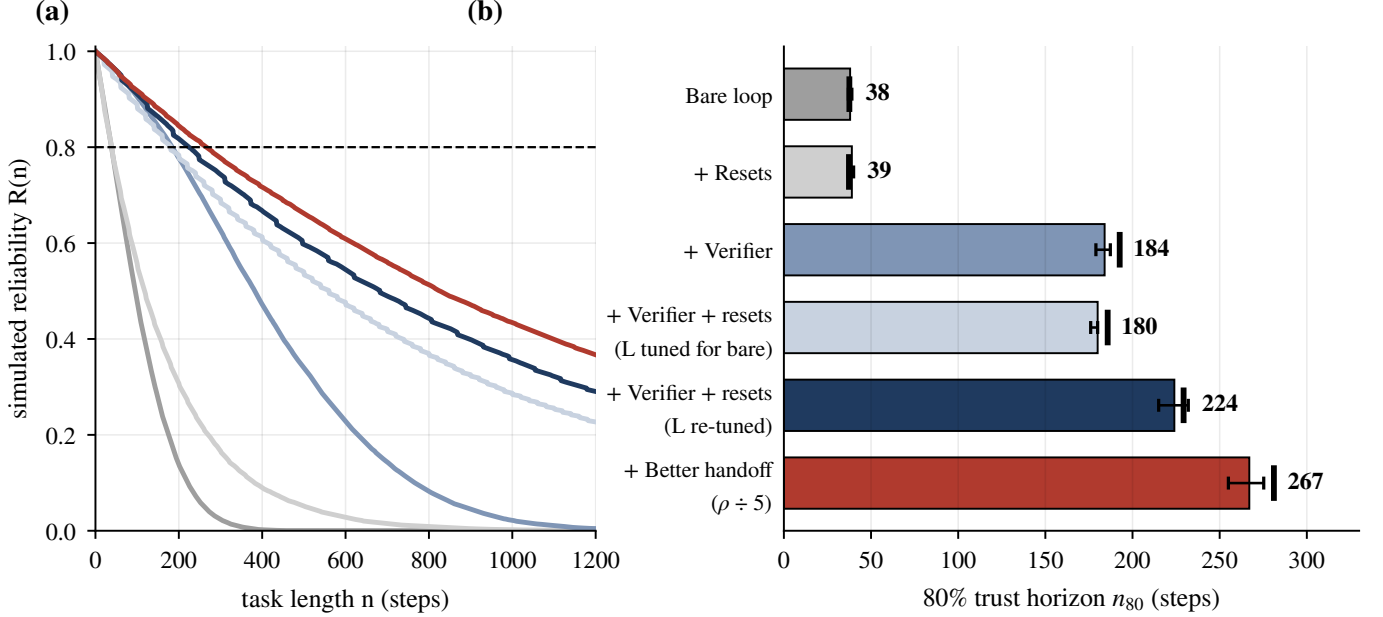


Fig. 12. Ablation of harness controls. (a) Simulated survival curves, shaded as in (b). (b) 80% trust horizon with 95% bootstrap intervals (20 000 tasks per condition); black ticks denote the prediction of (14).

- 5) Re-tune compaction or reset intervals whenever the sensor changes; the optimal interval scales as $1/\sqrt{\varepsilon}$.
- 6) In the memory-limited regime, invest in handoff fidelity; reducing ρ outperforms comparable investment in sensor recall.
- 7) Checkpoint verified progress to convert exponential rerun cost into linear cost, while recognizing that checkpoints do not substitute for sensors.

For organizations practicing site reliability engineering, these principles correspond to an error budget per task, reliability targets expressed in nines per step, and failure modes that are designed rather than discovered [2].

VII. PROPOSED EMPIRICAL VALIDATION

The model is useful only if its parameters can be measured on real agents and its predictions survive testing. This section specifies the hypotheses, the observations that would refute them, and a measurement protocol.

A. Hypotheses

Table VI lists six hypotheses with their refutation criteria.

B. Protocol

The protocol requires tasks whose intermediate states can be verified automatically, so that each step can be labeled correct or incorrect independently of the agent's own sensors (Fig. 14). Three task families are proposed: multi-stage data transformations with a verifiable table after each stage; sequential refactoring chains with a unit test per step; and stateful tool-use tasks, in the style of τ -bench, in which the database state after each action can be checked. Chaining short verified units makes task length a controlled variable, which existing benchmarks do not provide.

The design comprises two or three models, including at least one open-weights model; six task lengths from 4 to 128 steps; at least 30 task instances per length; at least eight repetitions per instance, for pass^k and the frailty test; and a factorial harness grid over sensor type, retry budget, and reset interval. Parameters are estimated on half of the task lengths and predictions are evaluated on the remainder. Specifically, ε_0 is estimated from step labels immediately after resets; γ from the slope of step error against context age; r and f from

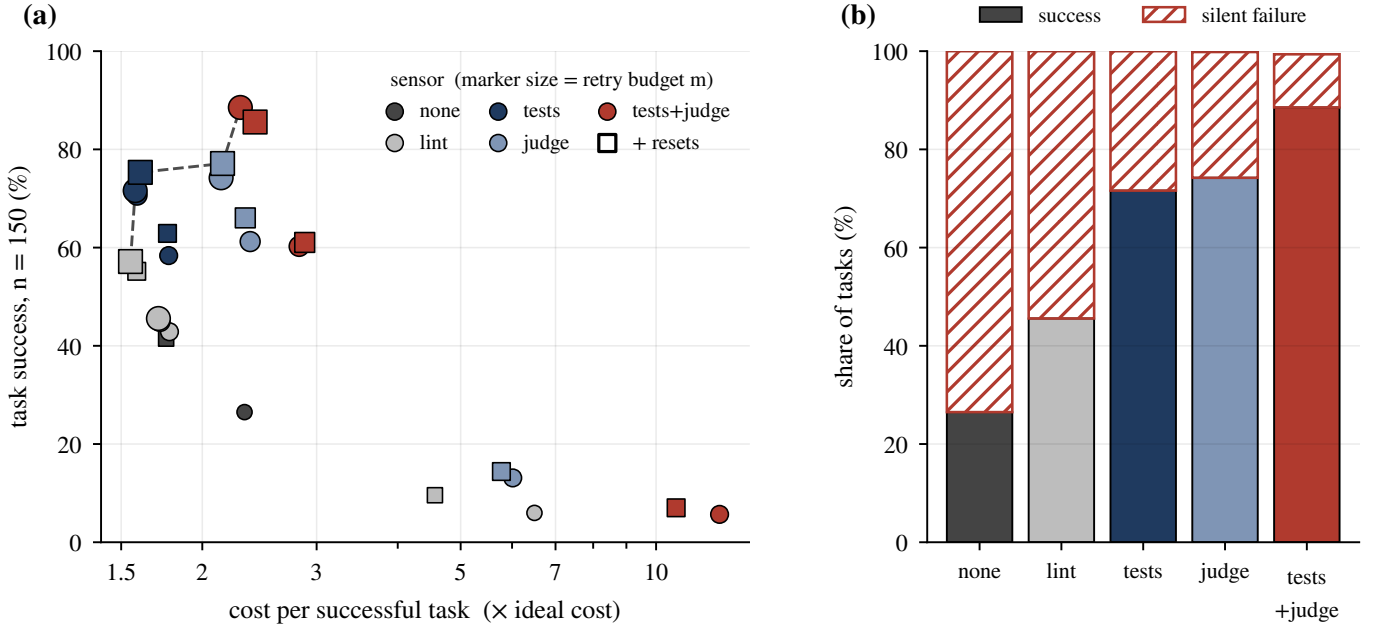


Fig. 13. Cost-reliability frontier for a 150-step task. (a) Success versus expected cost per successful task; dashed line: Pareto frontier. Configurations below 2% success, which include every sensor other than the linter with $m = 1$, are omitted. (b) Outcomes for the best configuration of each sensor without resets: success (solid) and silent failure (hatched). Sensor parameters are illustrative assumptions.

TABLE VI
FALSIFIABLE HYPOTHESES AND REFUTATION CRITERIA

	Hypothesis	Refuted if
H1	Within a task instance, per-step hazard is approximately constant over short horizons; departures are explained by degradation (increasing) and heterogeneity (apparent decrease).	Per-task survival curves from repeated trials exhibit a declining hazard comparable to the population curve.
H2	The declining population hazard in agent time-horizon data arises mainly from selection due to task heterogeneity rather than within-task learning.	A frailty model fits repeated-trial data significantly worse than a within-task learning model (likelihood-ratio test, $p < 0.01$).
H3	The measured gain in $n_{0.8}$ from a sensor equals $(1 - f)/(1 - r)$, with r and f estimated from labeled step outputs.	Measured gain deviates from prediction by more than 25% for two or more sensor types.
H4	With a sampled language-model judge, the silent-error rate on difficult steps increases with the retry budget; with a deterministic sensor it does not.	No increase in silent errors from $m = 2$ to $m = 8$ with the stochastic judge.
H5	The optimal compaction or reset interval scales as $1/\sqrt{\varepsilon}$: introducing a sensor with gain G shifts it by approximately $\sqrt{G}$.	The optimum does not shift, or shifts by less than half the predicted factor.
H6	Beyond sensor recall of approximately 0.95, reducing handoff loss increases $n_{0.8}$ more than an equal-cost increase in recall.	The converse holds in two or more task families.

sensor verdicts against ground-truth labels; ρ from failures on the first step after a reset that are attributable to missing state; and κ from the Beta fit to repeated trials (Proposition 3).

VIII. LIMITATIONS AND THREATS TO VALIDITY

Absence of real-agent measurements: All results are analytical or simulated, and the parameter values are illustrative. The value of the model depends on confirmation of its functional forms under the protocol of Section VII.

Definition of a step: METR measures task length in human minutes, whereas this model uses agent steps. The relation between the two is task-dependent and probably nonlinear; connections drawn to METR’s data concern the shape of the curves, not their units.

Independence: Errors may be correlated across steps (an incorrect plan affects many subsequent steps), and sensors may share the generator’s blind spots, reducing effective recall precisely where it matters. Propositions 2 and 5 relax independence in two specific forms; other dependence structures are not treated.

Linear degradation: The form $\varepsilon_0(1 + \gamma s)$ is the simplest assumption consistent with the evidence [30]; degradation may be nonlinear, task-dependent, or partially offset by adaptation to the environment.

Tension with declining hazards: The model predicts increasing per-task hazard from degradation, whereas population data exhibit declining hazard. Heterogeneity can mask the former, and deployed harnesses already manage context, but the tension

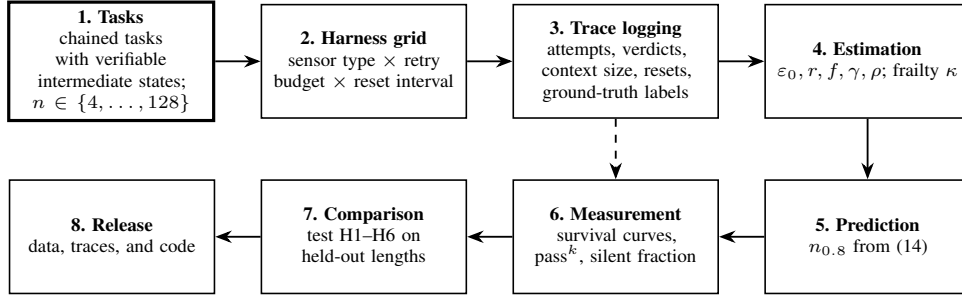


Fig. 14. Measurement protocol. Parameters estimated on one set of task lengths are used to predict survival on held-out lengths, providing an out-of-sample test of the model.

is unresolved; it is the subject of H1 and H2.

Literature figures: Harness-variance comparisons are observational; τ -bench values derive from the original release, whose tasks were subsequently revised; and several 2026 sources are preprints not yet peer reviewed. These figures motivate the model but do not test it.

Sensor parameters: The recall, false-alarm, and cost values in Section V-D are assumptions; their measurement is part of the proposed protocol.

IX. OPEN QUESTIONS

Six questions remain open. *Selection or learning:* is the declining hazard observed in agents a property of the agents or of the task distribution? *Step calibration:* can agent steps be mapped to human-equivalent time so that per-step reliabilities translate into time horizons? *Generator–judge correlation:* if a model and its judge share failure modes, effective recall on difficult steps may be far below measured recall. *Handoff loss:* can ρ be measured directly for compaction, summaries, progress files, and commits? *Harness transfer:* does a harness’s gain transfer across model families, or is part of the observed variance a model–harness interaction? *Tail reliability:* current data cannot discriminate between survival models whose implied 99% horizons differ by more than an order of magnitude; an experimental design capable of doing so is needed.

X. CONCLUSION

The gap between what agents can do and what they can be trusted to do follows from series reliability applied to a loop of fallible steps, compounded by heterogeneity across tasks and by degradation of context. Improved models raise all horizons proportionally but do not, alone, change the ratio between capability and trust; harness structure does. A reliability model with one parameter per harness component explains the observed ratio, shows that sensors multiply the trust horizon whereas checkpoints reduce the cost of failure, identifies the error-laundering risk of unbounded retries against stochastic verifiers, and predicts that once verification is strong, handoff fidelity becomes the limiting factor. Each prediction is testable with the protocol described.

APPENDIX A PROOFS

Proposition 1: Independent steps give $R(n) = (1 - \varepsilon)^n$. Setting $R(n) = \alpha$ gives $n_\alpha = \ln \alpha / \ln(1 - \varepsilon)$, and the ratio of two such expressions eliminates $\ln(1 - \varepsilon)$. The approximation uses $-\ln(1 - \varepsilon) = \varepsilon + O(\varepsilon^2)$. ■

Proposition 2: Conditional on λ , survival is $e^{-\lambda t}$. The Gamma Laplace transform gives $\mathbb{E}[e^{-\lambda t}] = (\beta/(\beta + t))^k$, and $h(t) = -\frac{d}{dt} \ln S = k/(\beta + t)$. Solving $S(T_\alpha) = \alpha$ gives $T_\alpha = \beta(\alpha^{-1/k} - 1)$. At $k = 1$, $T_{0.8}/T_{0.5} = 0.25$ and $S(t) = 1/(1 + t/T_{0.5})$. As $k \rightarrow \infty$ with fixed median, $\alpha^{-1/k} - 1 \sim \ln(1/\alpha)/k$, so the ratio tends to $\ln 1.25 / \ln 2$. ■

Proposition 3: By the Beta-function recurrence, $\mathbb{E}[\theta^k] = B(a+k, b)/B(a, b) = \prod_{j=0}^{k-1} (a+j)/(a+b+j)$, and $\text{pass}@k = 1 - \mathbb{E}[(1 - \theta)^k]$ by symmetry. Since $\text{Var}(\theta) = \mu(1 - \mu)/(a + b + 1)$ with $\mu = a/(a + b)$, $\kappa = 1/(a + b + 1)$. The fits in Table III solve $\mu = \text{pass}^1$ and $\mathbb{E}[\theta^4] = \text{pass}^4$ numerically. ■

Proposition 4: Each attempt independently terminates in state a , b , or c of (6). The step ends with an acceptance at attempt $j \leq m$ with probability $c^{j-1}(a + b)$; summing the geometric series yields (7), and the expected number of attempts is $\sum_{j=1}^m c^{j-1}$. Conditional on acceptance the error rate is $b/(a + b)$; expansion for small ε gives (8). ■

Proposition 5: On a stuck step every attempt is incorrect and is flagged independently with probability r , so it is accepted before the budget is exhausted with probability $1 - r^m$. The j th attempt occurs on a non-stuck step with probability c^{j-1} and yields a correct acceptance with probability a ; on a stuck step it occurs with probability r^{j-1} and yields a silent error with probability $1 - r$. Weighting silent errors by λ gives the stated marginal value, which is negative when $(c/r)^{j-1} < \lambda \varphi(1 - r)/[(1 - \varphi)a]$; taking logarithms with $c < r$ gives (10). ■

Proposition 6: Over one reset cycle, $-\ln R \approx \sum_{s=0}^{L-1} \varepsilon_0(1 + \gamma s) + \rho = \varepsilon_0 L + \frac{1}{2} \varepsilon_0 \gamma L(L - 1) + \rho$, using $-\ln(1 - x) \approx x$. Division by L yields (11). Setting $h'(L) = \frac{1}{2} \varepsilon_0 \gamma - \rho/L^2 = 0$ gives L^* , and $h(L^*) = \varepsilon_0(1 - \gamma/2) + \sqrt{2\varepsilon_0 \gamma \rho}$. ■

Proposition 7: Each segment attempt costs $L + v$ and succeeds with probability p^L ; the number of attempts is geometric with mean p^{-L} , and there are n/L segments. Minimizing $\ln(L + v) - \ln L + hL$ gives $h = 1/L - 1/(L + v)$, i.e., $L^2 + vL - v/h = 0$. ■

Equation (14): With a sensor, the accepted-error rate at context age s is $\tilde{\varepsilon}(1 + \gamma s)$ to first order, so Proposition 6

applies with ε_0 replaced by $\tilde{\varepsilon}$. Handoff loss is a silent error not observed by the step sensor and is not scaled by $1 - r$. Escalation contributes hazard $c^m \approx f^m$ per step. Summing the three hazards and applying (1) gives the result. ■

APPENDIX B SIMULATION DETAILS

The simulator implements Definition 1. At each step it operates on all surviving tasks simultaneously: it applies a reset and handoff loss if due, computes $\varepsilon(s)$, draws up to m attempts (stuck steps repeat their error), draws independent sensor verdicts, and records the first silent error or escalation. Task heterogeneity is introduced by drawing $\varepsilon_0 \sim \text{Gamma}(k, \varepsilon_0/k)$ per task. Bootstrap intervals resample tasks with 150 replicates. All experiments use fixed random seeds.

APPENDIX C NOTATION

Table VII summarizes the notation.

TABLE VII
NOTATION

Symbol	Meaning
$n, R(n)$	task length in steps; probability of completing n steps
n_α, n_α^*	trust horizon at reliability α ; its closed-form approximation
$\varepsilon_0, \tilde{\varepsilon}$	per-attempt error after a reset; silent-error rate after verification
r, f	sensor recall and false-alarm rate
m, φ	retry budget; probability that a step is stuck
γ, ρ	context-degradation rate; handoff loss per reset
L, L^*	reset interval and its optimum
G	verification gain $\varepsilon/\tilde{\varepsilon}$
h	hazard (per-step failure rate)
k	Gamma frailty or Weibull shape parameter
θ, κ	per-task success probability; intra-task correlation
v	sensor or checkpoint cost

ACKNOWLEDGMENT

In accordance with IEEE policy on the use of artificial intelligence-generated content, the author discloses that Claude (Anthropic) was used to assist with the literature search, mathematical derivations, simulation code, figure generation, and drafting of all sections of this manuscript. The author reviewed the content and takes responsibility for it. No external funding was received.

REFERENCES

- [1] P. D. T. O'Connor and A. Kleyner, *Practical Reliability Engineering*, 5th ed. Chichester, U.K.: Wiley, 2012.
- [2] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA, USA: O'Reilly Media, 2016.
- [3] R. Lopopolo, "Harness engineering: Leveraging Codex in an agent-first world," OpenAI Engineering, Feb. 2026, accessed: Sep. 23, 2026. [Online]. Available: <https://openai.com/index/harness-engineering/>
- [4] B. Böckeler, "Harness engineering for coding agent users," martinoflower.com, Apr. 2026, accessed: Sep. 23, 2026. [Online]. Available: <https://martinoflower.com/articles/harness-engineering.html>
- [5] J. Guo *et al.*, "From question answering to task completion: A survey on agent system and harness design," arXiv:2606.20683, Jun. 2026. [Online]. Available: <https://arxiv.org/abs/2606.20683>
- [6] Y. Zhang, J. Wang, Y. Ge, W. Xu, J. Hamm, and C. K. Reddy, "Stop comparing LLM agents without disclosing the harness," arXiv:2605.23950, May 2026. [Online]. Available: <https://arxiv.org/abs/2605.23950>
- [7] T. Kwa *et al.*, "Measuring AI ability to complete long tasks," arXiv:2503.14499v1, Mar. 2025, first version; the 50% and 80% horizons of Claude 3.7 Sonnet are reported in Sec. 4.2.1. [Online]. Available: <https://arxiv.org/abs/2503.14499v1>
- [8] —, "Measuring AI ability to complete long software tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 38, 2025, arXiv:2503.14499v4.
- [9] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, " τ -bench: A benchmark for tool-agent-user interaction in real-world domains," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025, arXiv:2406.12045.
- [10] FA&FO Specialist, "Claude Code: Figured out why /compact loses so much useful context, and a potential fix," Substack, Jan. 2026, accessed: Sep. 23, 2026. [Online]. Available: <https://faafospecialist.substack.com/p/claude-code-figured-out-why-compact>
- [11] G. Neubig, "Loop detection kills agents that are waiting on long-running processes," GitHub issue #5355, OpenHands/OpenHands, Dec. 2024, accessed: Sep. 23, 2026. [Online]. Available: <https://github.com/OpenHands/OpenHands/issues/5355>
- [12] MindStudio, "AI agent memory wall: Why agents fail at long-running jobs and how to fix it," MindStudio blog, Mar. 2026, accessed: Sep. 23, 2026. [Online]. Available: <https://www.mindstudio.ai/blog/ai-agent-memory-wall-long-running-jobs>
- [13] B. Böckeler and C. Ford, "Harness engineering and agent feedback: Exploring AI coding sensors," Thoughtworks Insights (blog), May 2026, accessed: Sep. 23, 2026. [Online]. Available: <https://www.thoughtworks.com/en-us/insights/blog/generative-ai/harness-engineering-agent-feedback-exploring-ai-coding-sensors>
- [14] E. Schluntz and B. Zhang, "Building effective agents," Anthropic Engineering, Dec. 2024, accessed: Sep. 23, 2026. [Online]. Available: <https://www.anthropic.com/engineering/building-effective-agents>
- [15] P. Rajasekaran, E. Dixon, C. Ryan, and J. Hadfield, "Effective context engineering for AI agents," Anthropic Engineering, Sep. 2025, accessed: Sep. 23, 2026. [Online]. Available: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- [16] K. Aizawa, "Writing effective tools for agents—with agents," Anthropic Engineering, Sep. 2025, accessed: Sep. 23, 2026. [Online]. Available: <https://www.anthropic.com/engineering/writing-tools-for-agents>
- [17] J. Young, "Effective harnesses for long-running agents," Anthropic Engineering, Nov. 2025, accessed: Sep. 23, 2026. [Online]. Available: <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- [18] A. Osmani, "Agent harness engineering," O'Reilly Radar, May 2026, accessed: Sep. 23, 2026. [Online]. Available: <https://www.oreilly.com/radar/agent-harness-engineering/>
- [19] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, Jul. 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [20] T. Ord, "Is there a half-life for the success rates of AI agents?" arXiv:2505.05115, May 2025. [Online]. Available: <https://arxiv.org/abs/2505.05115>
- [21] F. Hamilton, "Peto's paradox and the future of AI agents," gushamilton.github.io, Jan. 2026, updated Feb. 3, 2026. Accessed: Sep. 23, 2026. [Online]. Available: <https://gushamilton.github.io/lab/2026/01/23/petos-paradox-ai-agents/>
- [22] T. Ord, "Hazard rates for AI agents decline as a task goes on," tobyord.com, Feb. 2026, accessed: Sep. 23, 2026. [Online]. Available: <https://www.tobyord.com/writing/hazard-rates-for-ai-agents-decline>
- [23] S. Kapoor *et al.*, "Holistic agent leaderboard: The missing infrastructure for AI agent evaluation," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026, arXiv:2510.11977.
- [24] J. Huang *et al.*, "Large language models cannot self-correct reasoning yet," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024, arXiv:2310.01798.
- [25] J. Pan *et al.*, "Training software engineering agents and verifiers with SWE-Gym," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 267, 2025, pp. 47 717–47 737.
- [26] Y. Zhang *et al.*, "Small language models need strong verifiers to self-correct reasoning," in *Findings Assoc. Comput. Linguistics: ACL 2024*, Bangkok, Thailand, Aug. 2024, pp. 15 637–15 653.
- [27] J. Wei, "Asymmetry of verification and verifier's rule," jasonwei.net, Jul. 2025, accessed: Sep. 23, 2026. [Online]. Available: <https://www.jasonwei.net/blog/asymmetry-of-verification-and-verifiers-law>

- [28] A. Karpathy, “Verifiability,” [karpathy.bearblog.dev](https://karpathy.bearblog.dev/verifiability/), Nov. 2025, accessed: Sep. 23, 2026. [Online]. Available: <https://karpathy.bearblog.dev/verifiability/>
- [29] N. F. Liu *et al.*, “Lost in the middle: How language models use long contexts,” *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, 2024.
- [30] K. Hong, A. Troynikov, and J. Huber, “Context rot: How increasing input tokens impacts LLM performance,” Chroma, Tech. Rep., Jul. 2025, accessed: Sep. 23, 2026. [Online]. Available: <https://research.trychroma.com/context-rot>
- [31] J. W. Vaupel, K. G. Manton, and E. Stallard, “The impact of heterogeneity in individual frailty on the dynamics of mortality,” *Demography*, vol. 16, no. 3, pp. 439–454, 1979.
- [32] J. W. Young, “A first order approximation to the optimum checkpoint interval,” *Commun. ACM*, vol. 17, no. 9, pp. 530–531, 1974.
- [33] J. T. Daly, “A higher order estimate of the optimum checkpoint interval for restart dumps,” *Future Gener. Comput. Syst.*, vol. 22, no. 3, pp. 303–312, 2006.
- [34] Sierra Research, “ τ -bench: Leaderboard,” GitHub repository [sierra-research/tau-bench](https://github.com/sierra-research/tau-bench), tool-calling agents, pass¹–pass⁴. Accessed: Sep. 23, 2026. [Online]. Available: <https://github.com/sierra-research/tau-bench>
- [35] J.-M. P. Shackelford, “LLMBadRequestError from a poisoned tool_result permanently wedges conversations—no recovery path in Agent.step,” GitHub issue #5225, OpenHands/software-agent-sdk, Sep. 2026, accessed: Sep. 23, 2026. [Online]. Available: <https://github.com/OpenHands/software-agent-sdk/issues/5225>